

Universidade Técnica Do Atlântico

Instituto de Engenharias e Ciências do Mar

Sidnei Cruz
scruz@uta.cv



Estruturas de Dados Ligadas Especiais

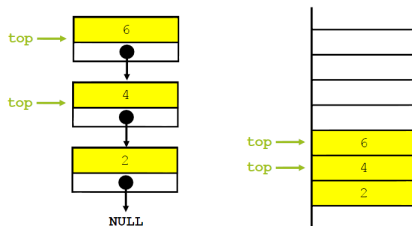
- Pilhas (STACKS)

- Uma das estruturas de dados ligadas mais simples é a pilha. Possivelmente por essa razão, é uma das estruturas de dados mais utilizada em programação, sendo inclusive implementada directamente pelo hardware da maioria das máquinas modernas.
- A ideia fundamental de uma pilha é que todo o acesso aos seus elementos é feito através do elemento que está no topo, ou seja o último elemento a ser introduzido. O único elemento que pode ser removido da pilha é também o elemento do topo.
- Isto faz com que os elementos da pilha sejam retirados na ordem inversa à ordem em que foram introduzidos: o primeiro que sai é o último que entrou.

Estruturas de Dados Ligadas Especiais

- Pilhas (STACKS)

- Para entendermos o funcionamento de uma pilha, podemos fazer uma analogia com uma pilha de livros. Se quisermos adicionar um livro à pilha colocamo-lo no topo. Para retirar um livro da pilha, retiramos o do topo. Ou seja, temos que retirar o livro do topo para aceder ao próximo livro.



- Duas operações elementares:

- **Push()** - Coloca elemento no topo da pilha.
- **Pop()** - Retira elemento do topo da pilha.

Estruturas de Dados Ligadas Especiais

- Pilha é uma estrutura linear de informações que permite acesso (operações) em somente uma extremidade.
- Por essa razão, a pilha é chamada de estrutura LIFO (last in /first out).

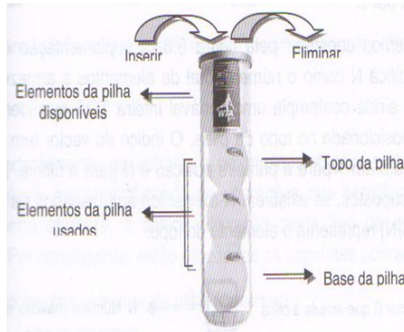


Figura: Funcionamento da estrutura Pilha adaptado no tubo de bolas de tênis

Estruturas de Dados Ligadas Especiais

- Pilha é uma estrutura linear de informações que permite acesso (operações) em somente uma extremidade.
- Por essa razão, a pilha é chamada de estrutura LIFO (last in /first out).



Figura: Funcionamento da estrutura Pilha - **Pilha Vazia**

Estruturas de Dados Ligadas Especiais

- Pilha é uma estrutura linear de informações que permite acesso (operações) em somente uma extremidade.
- Por essa razão, a pilha é chamada de estrutura LIFO (last in /first out).

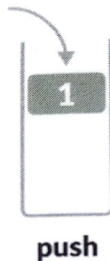


Figura: Funcionamento da estrutura Pilha - **push(1)**

Estruturas de Dados Ligadas Especiais

- Pilha é uma estrutura linear de informações que permite acesso (operações) em somente uma extremidade.
- Por essa razão, a pilha é chamada de estrutura LIFO (last in /first out).

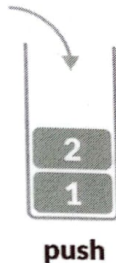


Figura: Funcionamento da estrutura Pilha - **push(2)**

Estruturas de Dados Ligadas Especiais

- Pilha é uma estrutura linear de informações que permite acesso (operações) em somente uma extremidade.
- Por essa razão, a pilha é chamada de estrutura LIFO (last in /first out).



Figura: Funcionamento da estrutura Pilha - **push(3)**

Estruturas de Dados Ligadas Especiais

- Pilha é uma estrutura linear de informações que permite acesso (operações) em somente uma extremidade.
- Por essa razão, a pilha é chamada de estrutura LIFO (last in /first out).



Figura: Funcionamento da estrutura Pilha - **pop()**

Estruturas de Dados Ligadas Especiais

- Pilha é uma estrutura linear de informações que permite acesso (operações) em somente uma extremidade.
- Por essa razão, a pilha é chamada de estrutura LIFO (last in /first out).

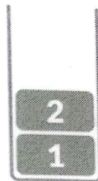


Figura: Estado da Pilha após as operações

Tipo de Dados Abstrato (TDA) - Stack/Pilha

- Definição da Interface

STACK.h

```
void STACKinit();  
  
int STACKempty();  
  
void STACKpush( Item );  
  
Item STACKpop();
```

TDA Stack/Pilha - Implementação com Lista

STACK.c

```
#include <stdlib.h>
#include "Item.h"
#include "STACK.h"

struct STACKnode{

    Item item;
    struct STACKnode* next;
};

typedef struct STACKnode* link;

static link head;
```

ADT Stack - Implementação com Lista

STACK.c

```
link NEW(Item item, link next){

    link x = (link)malloc(sizeof(STACKnode));
    x->item = item;
    x->next = next;
    return x;
}

void STACKinit(){

    head = NULL;
}

int STACKempty(){

    return head == NULL;
}
```

ADT Stack - Implementação com Lista

STACK.c

```
void STACKpush(Item item){  
    head = NEW(item, head);  
}  
  
Item STACKpop() {  
    Item item = head->item;  
    link t = head->next;  
    free(head);  
    head = t;  
    return item;  
}
```

Estruturas de Dados Ligadas

- **Filas (QUEUES):**

- Outra estrutura de dados bastante utilizada é a fila.
- A diferença entre uma fila e uma pilha é a ordem de saída dos elementos: enquanto que na pilha "o último elemento a entrar é o primeiro a sair", na fila "o primeiro a entrar é o primeiro a sair" (a sigla **FIFO** - First In First Out - é usada para descrever esta estratégia).
- Basicamente só podemos inserir um novo elemento no final da fila e só podemos retirar um elemento do início.

Estruturas de Dados Ligadas

- **Filas (Continuação)**

- A estrutura de uma fila é análoga ao conceito de fila que usamos no nosso dia a dia: quem entra primeiro numa fila é o primeiro a ser atendido (a sair da fila).

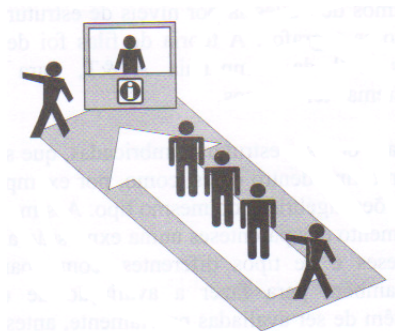


Figura: Fila de espera

Estruturas de Dados Ligadas

- **Filas (Continuação)**

- A estrutura de uma fila é análoga ao conceito de fila que usamos no nosso dia a dia: quem entra primeiro numa fila é o primeiro a ser atendido (a sair da fila).

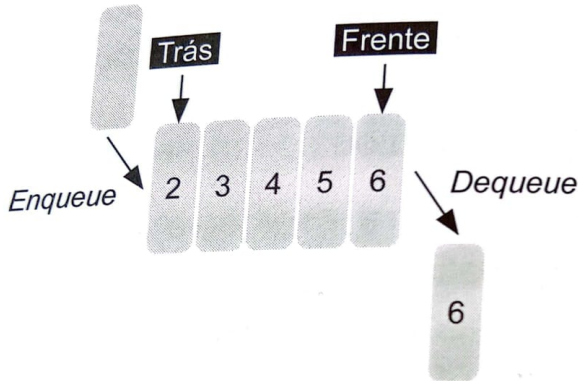


Figura: Funcionamento da estrutura Fila

TDA - QUEUES (FIFO)

- Definição da Interface

QUEUE.h

```
void QUEUEinit();
```

```
int QUEUEempty();
```

```
void QUEUEput(Item);
```

```
Item QUEUEget();
```

Implementação do TDA FIFO (Listas)

QUEUE.c (cont.)

```
#include <stdlib.h>
#include "Item.h"
#include "QUEUE.h"

typedef struct QUEUENode* link;

struct QUEUENode {
    Item item;
    link next;
};

static link head, tail;
```

Implementação do TDA FIFO (Listas)

QUEUE.c (cont.)

```
void QUEUEinit(){  
  
    head = NULL;  
    tail = NULL;  
}  
  
int QUEUEempty(){  
  
    return head == NULL;  
}  
  
link NEW(Item item, link next){  
  
    link x = malloc(sizeof(struct QUEUENode));  
    x->item = item;  
    x->next = next;  
    return x;  
}
```

Implementação do TDA FIFO (Listas)

QUEUE.c (cont.)

```
void QUEUEput(Item item){  
  
    if (head == NULL) {  
        head = (tail = NEW(item, head));  
        return;  
    }  
    tail->next = NEW(item, tail->next);  
    tail = tail->next;  
}
```

```
Item QUEUEget(){  
  
    Item item = head->item;  
    link t = head->next;  
    free(head);  
    head = t;  
    return item;  
}
```

Estruturas de Dados Ligadas

- Exemplos de utilização das pilhas e filas:
 - Determinar se uma cadeia de caracteres é uma capicua:
 - Exemplos:
 1. RADAR
 2. 123454321
 - Colocar cada caracter que compõe a cadeia numa pilha e numa fila.
 - Ao retirar os caracteres da pilha eles virão invertidos e ao retirar da fila eles virão pela mesma ordem pela qual foram introduzidos. Basta compará-los para ver se são iguais.

Estruturas de Dados Ligadas

- A pilha é habitualmente utilizada para avaliar expressões matemáticas:
- **Um exemplo simples:** verificar se numa determinada expressão existe coerência nos parêntesis utilizados:

$$((A + B * (C + 2)) * (D + E))$$

- **Solução:** Utilizar a pilha para verificar se o número de parênteses está correcta. Por cada '(' colocar um item na pilha e por cada ')' retirar um item da pilha. Se sempre que aparecer um ')' existir um valor na pilha e se no final a pilha estiver vazia, então a expressão tem os parêntesis correctos.